

EXPERIMENT: BACKWARD CHAINING

AIM

To understand and implement the concept of backward chaining in PROLOG using facts, rules, and queries.

OBSERVATION

1. Medical Diagnosis

Problem: Determine whether a person has flu based on fever and cough symptoms.

Program:

```
% Facts
has_fever(john).
has_cough(john).

% Rule
has_flu(X) :- has_fever(X), has_cough(X).

% Query
% ?- has_flu(john).
```

Output:

true.

Observation:

PROLOG starts with the goal `has_flu(john)` and works backward by checking the conditions in the rule. Since both `has_fever(john)` and `has_cough(john)` are true, the query succeeds.

2. Student Eligibility Checker

Problem: Determine whether a student is eligible based on passing Mathematics and Science.

Program:

```
% Facts
passed(alice, maths).
passed(alice, science).

% Rule
eligible(X) :- passed(X, maths), passed(X, science).

% Query
% ?- eligible(alice).
```

Output:

```
true.
```

Observation:

PROLOG starts with the goal `eligible(alice)` and checks whether Alice satisfies both conditions in the rule. Since Alice has passed both Mathematics and Science, the query succeeds.

3. Travel Planner

Problem: Determine whether a person can travel based on having a ticket and visa.

Program:

```
% Facts
has_ticket(james).
has_visa(james).

% Rule
can_travel(X) :- has_ticket(X), has_visa(X).

% Query
```

```
% ?- can_travel(james).
```

Output:

true.

Observation:

PROLOG starts with the goal `can_travel(james)` and works backward to verify whether James has both a valid ticket and visa. Since both facts are available, the query succeeds.

RESULT

Thus, the concept of backward chaining was successfully implemented in PROLOG using medical diagnosis, student eligibility checking, and travel planning examples. The given queries were successfully evaluated and the expected results were obtained.